Manwe Castro
EEE 202 Circuits I Lab
01 May 2026

Final Design Project Report

In order to adhere to the needs of Sparky Solar, the group was tasked with the objective of designing a solar panel which satisfied the needs of Melanie Waters, the Sparky Solar CEO. The project focused on constructing a small, portable solar panel power meter which would be readily available, be affordable, include a compact design, and operate under a high maximum temperature. These four criteria for the power meter had to be taken into consideration throughout the entire process. The inspiration behind the project was to make it easier for technicians to read the voltage, current, and power without having to calculate the values by hand. The primary features of the design included measuring the current through a shunt resistor, using an op amp to enhance the small voltages, and then implementing arduino code to calculate and display the values on an LCD digital display. A basic schematic as well as design requirements were provided. However, it was up to the students to select resistor values and other design decisions to make the power meter as efficient as possible.

Design Intent and Stakeholder Considerations

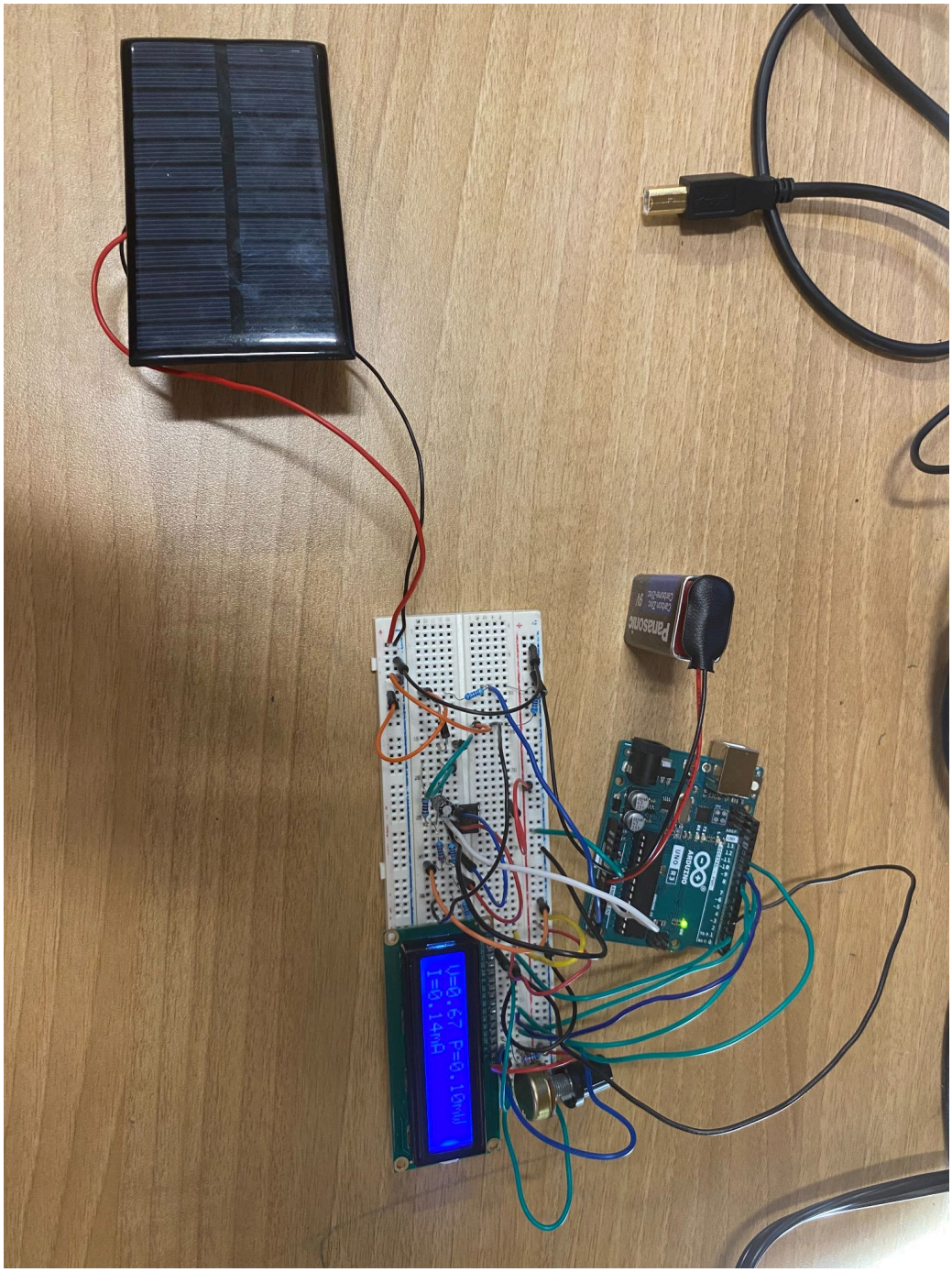
Our group built this meter with a prioritization for field technicians. They needed a dependable way to check small solar setups that was quick and did not require the use of outside tools or calculations. That focus on portability drove a lot of component choices, especially sticking to standard resistor kits and keeping the part count low. Testing the panel showed how wildly the output can change: 6.83 V in direct sun, dropping to 1.3 V in the shade, and barely 0.19 V under lab lights. Since those peaks go past the Arduino's 5 V analog limit, we knew we had to scale the signals down first.

Scaling this up for commercial arrays would introduce a lot of new challenges. Outdoors, the housing would need proper weather sealing and corrosion-proof terminals. Higher system

voltages would call for isolation amps or step-down transformers, and instead of a simple LCD, the meter would probably need to log data or send readings over Modbus or wireless links. Utility customers also expect lab-grade accuracy for billing, so regular calibration against certified gear would be mandatory. While buying parts in bulk would cut unit costs, safety certifications and compliance testing would drive up development time. Heat management would matter a lot too, since long stretches of direct sun can warp or degrade components if they aren't properly rated or vented.

Technical Design and Implementation

The meter works with three main components; voltage sensing, current sensing, and the Arduino for processing. We wired the panel to a 1 kΩ load resistor and added a diode to stop any reverse current. From there, R1 and R2 form a voltage divider that steps the panel voltage down to a safe level for Arduino pin A1. To track current, we placed a 50 Ω shunt in series with the load. The tiny voltage drop across it gets fed into a non-inverting op-amp, which boosts the signal before it hits analog pin A0. The Arduino reads both pins, applies the reverse math to get real-world values, multiplies voltage by current for power, and refreshes the LCD every second.



We picked parts based on what Sparky Solar actually had in stock and what kept costs down. For the divider, we went with resistor set 2, settling on 47 kΩ for R1 and 100 kΩ for R2. Plugging the worst-case 6.83 V into the divider formula (Vout = Vin × R2/(R1 + R2)) gives about 4.64 V at the pin. That leaves enough buffer under the 5 V limit without losing ADC

resolution. Both resistors stay well within their 0.25 W ratings too; R2 only dissipates around 0.215 mW at max voltage, calculated as $(4.64\ V)^2/100\ k\Omega$.

Getting the op-amp gain right took some back-and-forth. At full sun, the load pulled 6.76 mA, which dropped about 338 mV across the 50 Ω shunt. we needed to scale that up without clipping the Arduino's 5 V input, so we chose 22 kΩ for Ri and 100 kΩ for Rf. That gives a gain of 5.55 (1 + Rf/Ri), pushing the max signal to roughly 1.87 V. We made sure to use a rail-to-rail op-amp so it could swing close to the supply rails, which really helps when you're trying to read tiny shunt voltages without losing detail.

Every choice made throughout the design came with a compromise, one of which being the bigger divider resistors save power but pick up more noise and leakage. The 5.55 op-amp gain was a compromise, being sensitive enough for low currents, but low enough to avoid saturating when the sun hits hard. We also used a benchtop DC supply indoors to mimic sunlight. It wasn't perfectly realistic, but it gave us steady numbers to debug against before we ever took the circuit outside for real-world testing.

Validation and Iteration

When we compared our hand calculations to what the circuit actually did, the numbers were close but not perfect, which is pretty standard for breadboard builds. The voltage divider should have output 4.64 V from a 6.83 V input, but our multimeter read between 4.58 V and 4.62 V. That 0.4 to 1.3% drift lines up with the ±5% tolerance on the resistors we grabbed. The op-amp showed similar behavior: we calculated a gain of 5.55, but testing gave us 5.48 to 5.52. Those small shifts come down to resistor tolerances and the op-amp's own input offset voltage.

The clearest mismatch showed up during full-sun testing. The Arduino's serial monitor reported 6.71 V, while our handheld meter read 6.83 V. We tracked it down to the Arduino's 5 V rail not being exactly 5.00 V, which threw off the ADC conversion math in our code. We pulled up a calibrated meter, measured the actual 5 V pin at 4.97 V, and swapped that value into the

sketch. The error dropped below 0.2% after that. It was a good reminder to always measure your actual board voltages instead of trusting the nominal specs.

We also ran into a grounding glitch during assembly that made the LCD flicker and spit out jumping numbers. We started by checking all the power rails, then scoped the signal lines to see where things were breaking down. Turns out the op-amp stage and the Arduino didn't share a common ground. Once we ran a jumper from the op-amp's ground rail straight to the Arduino's GND pin, the readings locked in. It reinforced how crucial a solid ground reference is when mixing analog and digital signals on the same board.

Professional Growth and Skills Development

Working through this build really sharpened two things: how we troubleshoot circuits and how we bridge the gap between simulation and real hardware. When the LCD started acting up, we made a point not to just swap parts randomly. Instead, we checked the power rails first, traced the signal path step-by-step with a scope, and eventually caught the floating ground. Taking that structured route saved us hours and kept us from burning through spare components.

We also got a lot better at validating our TinkerCAD models against actual bench results. We ran simulations for each component before wiring anything, but the real circuit always threw a slight difference. Those mismatches forced us to look into things like actual resistor tolerances, stray capacitance on the breadboard, and how our multimeter's input impedance affects readings. The op-amp gain drift, for instance, had us double-checking the datasheet and recalculating with worst-case tolerance bands. Going back and forth between simulating, building, and measuring led us to the conclusion that simulations are great for catching big mistakes, but you still have to test the physical board to know if it actually works.

Conclusion

The final meter hits all the targets Sparky Solar laid out. With the panel, it tracked voltages from 0.19 V up to 6.83 V and currents between 0.191 mA and 6.76 mA as lighting

changed. The voltage divider and op-amp did their job conditioning the signals for the Arduino, and the LCD gave clean, live readouts of voltage, current, and power. We kept parts cheap and easy to source without sacrificing accuracy. The small gaps between our simulated results and bench readings came down to normal part tolerances and real-world lighting shifts, nothing unexpected. Overall, this project tied together circuit theory, hands-on assembly, and troubleshooting into something that actually works, which is exactly the kind of practice that translates to real engineering work.